IRCTC: Cloud Scaling for Railway Ticket Booking and Tatkal Spikes

IT457 Cloud Computing | Week 2 Report | Group 40

Aarya Shah (202301036) • Visha Sitapara (202301414) • Krish Dhola (202301189)

Abstract

Tatkal booking creates a predictable burst of demand at 10:00 for AC classes and 11:00 for non-AC classes. IRCTC and CRIS respond by filtering unwanted traffic and improving reservation-system capacity. Public sources describe several protections and a new cloud-enabled Passenger Reservation System (PRS), but do not confirm the new system's cloud provider or complete internal architecture.

Our project will model the scaling problem with a small booking API behind a load balancer and an auto-scaling group. We will compare fixed capacity, reactive scaling and scheduled pre-scaling, while checking that concurrent bookings never sell more seats than exist. The proposed PoC is separate from IRCTC's production system.

Key facts at a glance

Item	What the sources say
Current PRS capacity	About 32,000 bookings and 4 lakh enquiries per minute [1][3].
New PRS announced target	More than 1.5 lakh bookings and 40 lakh enquiries per minute; a target, not a demonstrated peak result [1][3].
Traffic controls	CDN, anti-bot protection, Aadhaar / OTP checks and an agent blackout during the first 30 minutes of Tatkal [2][5][6].
Evidence boundary	No public confirmation found for the new PRS cloud provider, database engine or full scaling policy.

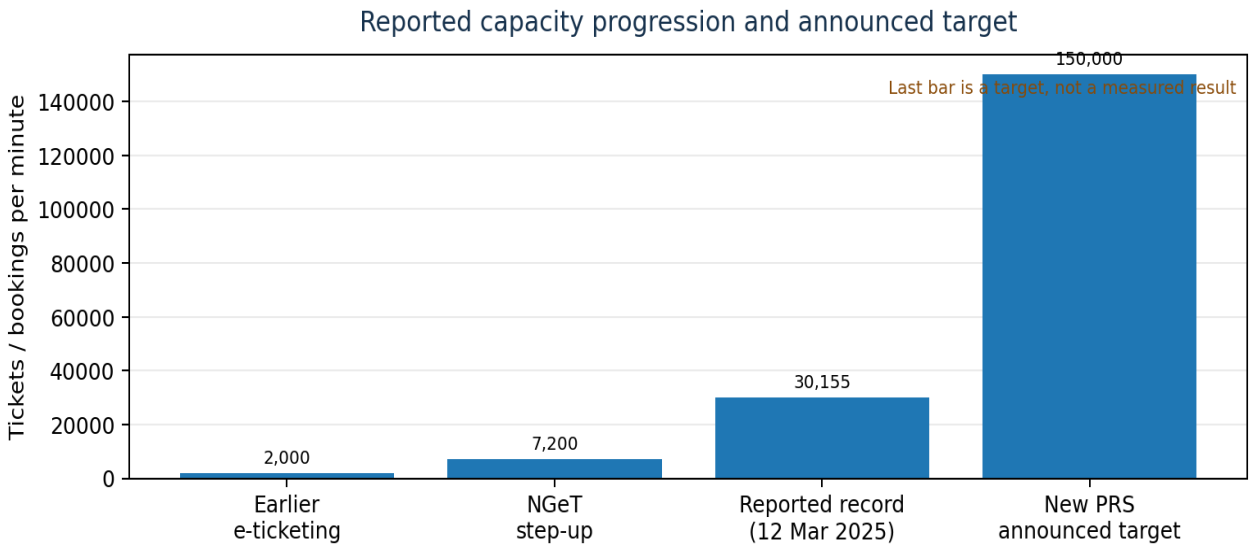


Figure 1. Historical capacity figures and the announced target. The target is clearly distinguished from measured results.

1. Case background

CRIS develops and maintains the Passenger Reservation System (PRS) and the NGeT online e-ticketing platform. The PRS handles reservation data such as seat availability, waitlists and bookings; NGeT provides the online booking channel. IRCTC operates the website and apps.

System / actor	Role
CRIS	Develops and maintains PRS and NGeT [3][4].
PRS / CONCERT	Reservation engine for seats, waitlists, RAC and charts [3][4].
NGeT	Online e-ticketing platform launched in 2014 [3][5].
IRCTC	Operates the website and mobile apps [5].
Ministry of Railways	Sets policies such as Tatkal Aadhaar / OTP requirements [1][2][6].

Scale matters: from June 2025 to June 2026, 65.08 crore reserved tickets were booked through PRS, and 57.90 crore (89%) were online [3]. IRCTC reported an average of 13.88 lakh tickets per day in FY2024–25 [5]. These figures use different reporting periods and measures.

2. The scaling problem

- Concentrated demand: bookings arrive in a short window instead of being spread evenly across the day.
- Unwanted traffic: IRCTC reports that bot mitigation blocked about 30% of traffic at the edge [5].
- Shared seat inventory: more servers cannot create more seats. Concurrent booking updates must remain correct.

Capacity increased from about 2,000 bookings per minute in an earlier system to 7,200 and later over 30,000 in NGeT. IRCTC reported a record of 30,155 tickets in one minute on 12 March 2025. The new PRS target is over 1.5 lakh bookings per minute, but this remains a target [1][3][5].

3. How the system evolved

Date / period	Milestone
1986	PRS introduced; CRIS established [3].
2002 / 2014	Internet booking begins; NGeT launched [3][5].
2010	Current PRS generation deployed on Itanium servers and OpenVMS [3].
Feb 2025	CDN and anti-bot protection deployed on NGeT [5].
Jun–Dec 2025	New PRS announced; Aadhaar and OTP measures phased in [1][2][6].
Jul–Aug 2026	New website beta and phased migration to the new PRS reported [3][7].

4. Existing architecture: what is publicly documented

Figure 2 is a simplified view of the layers described in official public sources. It is not a complete network diagram.

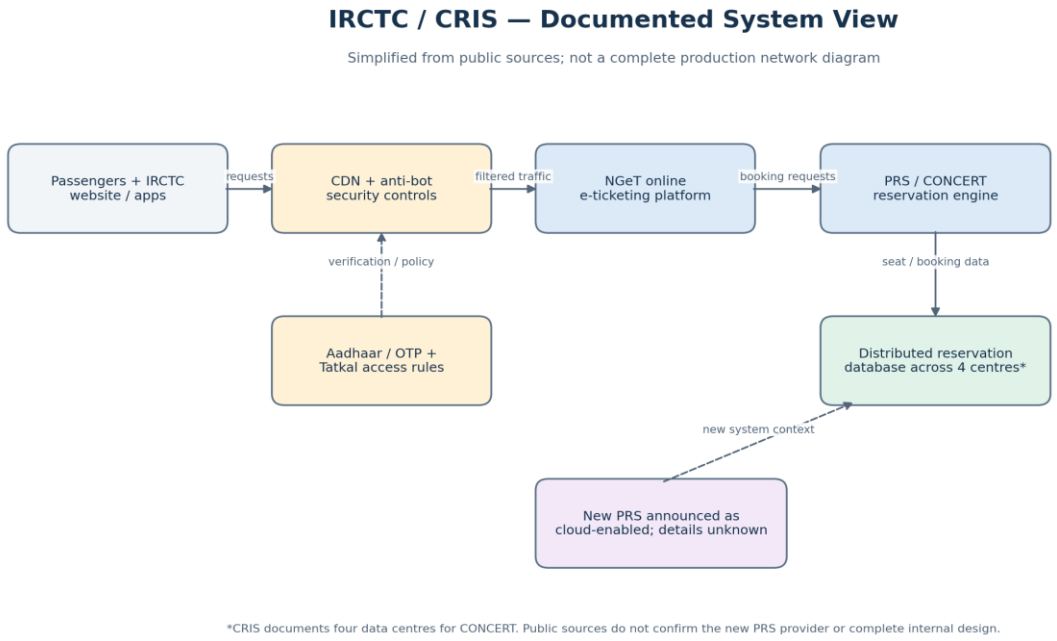


Figure 2. Documented IRCTC / CRIS system view; the new PRS internals remain unconfirmed.

Main layers

- Reservation engine: CRIS describes CONCERT as a mission-critical, three-tier transaction system with a distributed database and four data centres [4].
- Online platform: NGeT is hosted at a CRIS data centre in New Delhi. IRCTC also reports a budget for disaster recovery [5].
- Edge and security: CDN, anti-bot protection, firewalls and other controls reduce unwanted traffic before it reaches the booking system [2][5].

Important limitation: the available sources do not establish that the current booking path runs on a public cloud. A third-party security case study is not enough to confirm the new PRS provider or architecture [9].

5. The response: reduce demand and add capacity

Measure	Purpose / reported detail
CDN and anti-bot	Offloads repeated traffic and blocks suspicious requests; deployed on NGeT in February 2025 [5].
Aadhaar and OTP	Adds identity verification for online Tatkal bookings [2][6].
Agent blackout	Agents cannot book during the first 30 minutes of each Tatkal window [6].
Account clean-up	Suspicious user IDs and email domains were deactivated or blocked [2][5].
New PRS	Cloud-enabled system announced with targets of over 1.5 lakh bookings and 40 lakh enquiries per minute [1][3].

The main lesson is that filtering unnecessary requests can be as important as adding servers. However, higher system capacity does not guarantee a seat on a popular train, and public sources do not provide independent Tatkal latency or error-rate measurements.

6. Outcomes and evidence quality

Reported outcome	Evidence status
About 3.02 crore suspicious IDs deactivated	Official Ministry release [2].
30,155 tickets in one minute, 12 March 2025	Company-reported record in IRCTC Annual Report [5].
Tatkal availability time improved on some trains after OTP rollout	Official count-based result; it does not quantify the size of the time improvement [2].
1.5 lakh bookings per minute	Announced target only; not proof of achieved peak performance [1][3].

7. What the case teaches about cloud scaling

Concept	Lesson for our exercise
Horizontal scaling	Use several identical API instances behind one load balancer.
Demand shaping	Test rate limiting or a waiting queue, not just adding servers.
Scheduled scaling	Compare reactive autoscaling with capacity prepared before a known burst.
Shared-state correctness	Keep seat inventory in one shared store and use atomic booking updates.
Monitoring and cost	Track latency, errors, instance count, scaling events and cost per run.

8. Proposed proof-of-concept architecture

The design below is our own project architecture, not IRCTC's production design.

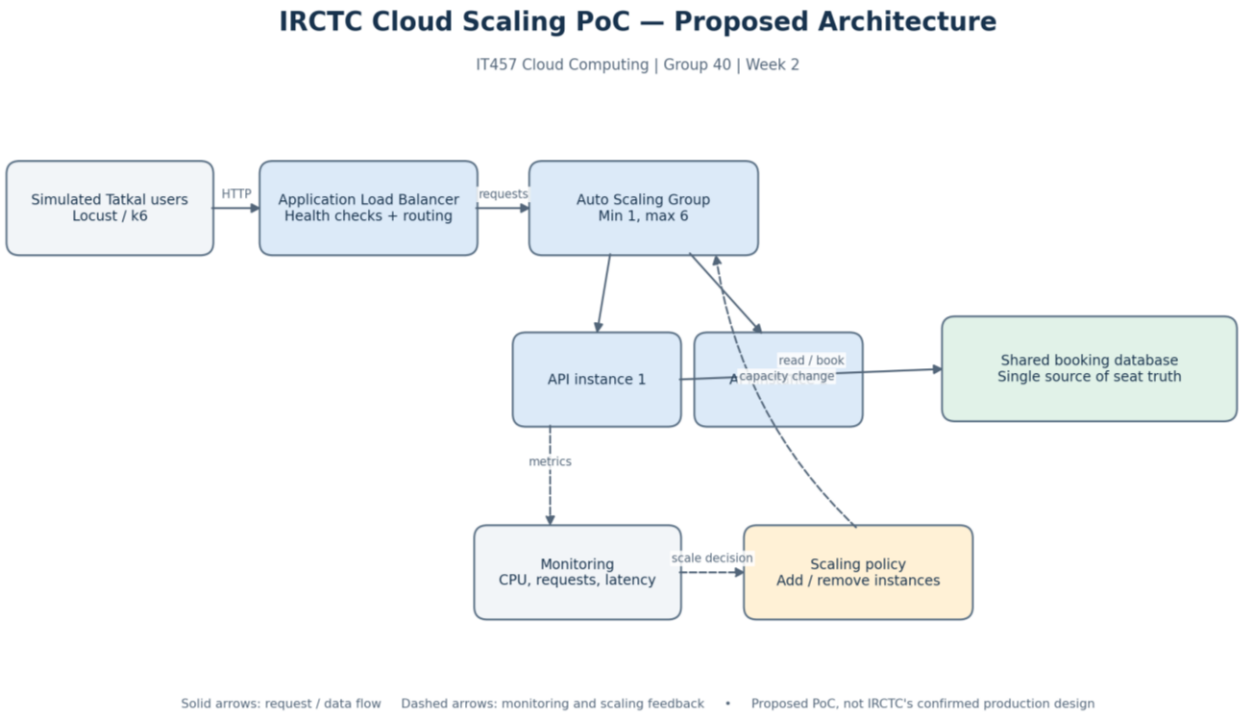


Figure 3. Proposed PoC: requests pass through the load balancer to scalable API instances; monitoring informs the scaling policy.

9. Exercise plan

Objective:

Measure how a load balancer and autoscaling respond to a Tatkal-style burst. The app will expose GET /health, GET /availability and POST /book. Seat inventory will be stored centrally; booking must use an atomic update to prevent overselling.

Proposed platform:

The preferred platform is AWS, using an Application Load Balancer, EC2 instances in an Auto Scaling Group, a shared data store and CloudWatch monitoring. The starting instance range is one to six small instances, subject to available credits and the group's final configuration. Another cloud platform can be used if it provides equivalent services and a public URL, but the group should settle on one platform before implementation.

Planned experiments:

Run	Setup	Question
R0	One fixed instance; ramp the load	Where does one instance saturate?
R1	Three fixed instances; same ramp	How does fixed capacity change latency?
R2	Autoscaling from 1 to 6 instances	How long does scaling take to help?
R3	Sharp burst in about 10 seconds	What happens before new instances are ready?
R4	Pre-warm capacity before the burst	Does scheduled scaling reduce the latency spike?
R5 (optional)	Burst with rate limiting / queue	Does demand shaping protect the API?
C1	Many concurrent bookings for few seats	Are exactly the available seats sold?

Load profile and metrics

- Warm-up: 10 virtual users for 2 minutes; ramp up over 10 minutes; burst for 5 minutes; cool down for 5 minutes.
- Initial workload assumption: 90% availability checks and 10% bookings. This is a test setting, not an IRCTC statistic.
- Record p50 / p95 / p99 latency, requests per second, error rate, healthy instance count, CPU and scaling timestamps.
- Main output: response time versus instance count, plus a timeline for each run.

Cost and completion

Preferred setup: an Application Load Balancer in front of an Auto Scaling group (minimum 1, maximum 6 small instances). The cloud provider and region will be selected by the group. Alternatives are acceptable if they support load balancing, autoscaling and a public URL.

Estimate cost from instance-hours, load-balancer charges and data transfer using the selected provider's calculator. No price is quoted until the region and service are chosen.

Expected hypotheses to test

1. Response time and errors rise once a fixed instance reaches its limit.
2. Auto-scaling can restore performance, but new instances take time to become ready.
3. For a predictable burst, scheduled pre-scaling may perform better than relying only on reactive scaling.

These are hypotheses about our PoC, not claims about IRCTC.

Risks and controls

The load generator itself may become a bottleneck, so its CPU will be monitored and it will run separately from the application where possible. Scaling cooldowns and instance start-up times will be recorded. Runs will be short because cloud credits are limited, and testing will target only our own deployment.

10. Team responsibilities

Member	Primary ownership	Shared deliverables
Aarya Shah (202301036)	Booking API, shared database integration, atomic booking logic and cloud deployment / live URL.	Co-design architecture; run correctness tests; review results; write the API and deployment sections.
Visha Sitapara (202301414)	Load-test scenarios in Locust / k6, autoscaling configuration, monitoring and performance charts.	Co-design architecture; run experiments; review results; write the experiment and analysis sections.
Krish Dhola (202301189)	IRCTC / CRIS case research, source verification, cost model and evidence tracking.	Build the run matrix and cost table; test the deployment; write case-study and evidence sections.

Shared accountability

- All three members review the architecture and verify that production facts are clearly separated from the proposed PoC.
- Record tasks and decisions in GitHub issues / pull requests, and commit individual work regularly. Commits support the record of contribution but are not the only evidence.

Suggested milestones

Week	Milestone
Week 3	Deploy API; finish R0–R5 and C1; save logs and settings.
Week 4	Generate charts; complete cost table; update report with measured results.
Week 5	Review limitations, and prepare for viva.

11. Limitations and open questions

- The new PRS cloud provider, database engine and full internal architecture are not public in the sources reviewed.
- The announced capacity numbers are targets; they should not be presented as measured peak performance.
- The PoC is a small-scale model. Its results demonstrate scaling behaviour, not IRCTC's actual performance.
- Revenue figures are not infrastructure costs; no verified production cost estimate is included.

12. Conclusion

IRCTC's case shows that high-traffic booking requires both demand management and sufficient application capacity. CDN and bot protection reduce avoidable traffic, while the new PRS is intended to increase reservation capacity. For our project, a small cloud PoC with a load balancer, auto-scaling application instances, shared booking storage and monitoring is a practical way to test the underlying cloud concepts. The experiments will focus on latency, errors, scaling delay, booking correctness and cost, with all results clearly separated from claims about IRCTC's production system.

Appendix: short glossary

PRS — Passenger Reservation System. • CONCERT — nationwide reservation application. • NGeT — Next Generation e-Ticketing. • CDN — content delivery network. • Autoscaling — automatically adding or removing instances. • p95 latency — the response time that 95% of requests are at or below.

References

1. [1] Ministry of Railways, PIB. “New Passenger Reservation System capable of generating over 1.5 lakh rail tickets per minute”, 29 Jun 2025. https://www.pib.gov.in/PressReleseDetailm.aspx?PRID=2140614&reg=3&lang=2
2. [2] Ministry of Railways, PIB. “Indian Railways strengthens ticket reservation system with robust cyber security”, 11 Dec 2025. https://www.pib.gov.in/PressReleasePage.aspx?PRID=2202197&lang=2&reg=3
3. [3] Ministry of Railways, PIB. “Four Decades of Passenger Reservation in Indian Railways”, 6 Aug 2026. https://www.pib.gov.in/PressNoteDetails.aspx?NoteId=159450&ModuleId=3&reg=48&lang=1
4. [4] Centre for Railway Information Systems. PRS project page (CONCERT, NGeT). https://www.cris.org.in/loadpage?page=proPRS
5. [5] IRCTC. Annual Report 2024–25. https://nsearchives.nseindia.com/annual_reports/AR_27340_IRCTC_2024_2025_A_07082025185707.pdf
6. [6] Lok Sabha Unstarred Question 2787, answered 6 Aug 2025 (Aadhaar authentication for Tatkal). https://sansad.in/getFile/loksabhaquestions/annex/185/AU2787_Ak5anh.pdf?source=pqals
7. [7] Business Today. “Indian Railways’ new ticket booking system will be 5x faster”, 11 Aug 2026. https://www.businesstoday.in/latest/trends/photo/indian-railways-new-ticket-booking-system-will-be-5x-faster-heres-whats-changing-for-irctc-users-548374-2026-08-11
8. [8] Punekar News. “IRCTC website revamp announced after RTI exposes Rs 2,619 crore convenience fee earnings”, 17 Jun 2026. https://www.punekarnews.in/irctc-website-revamp-announced-after-rti-exposes-rs-2619-crore-convenience-fee-earnings/
9. [9] CloudThat. “Strengthening defenses against common web exploits and malicious IPs with AWS security services for IRCTC” (undated). https://www.cloudthat.com/resources/case-study/strengthening-defenses-against-common-web-exploits-and-malicious-ips-with-aws-security-services-for-irctc
10. [10] Zee News. “Indian Railways spends Rs 1,000 cr to revamp 40-year-old IRCTC reservation system”, 16 Mar 2026. Cost figure is unconfirmed in the report.